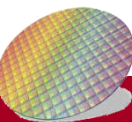




成功大學

National Cheng Kung University

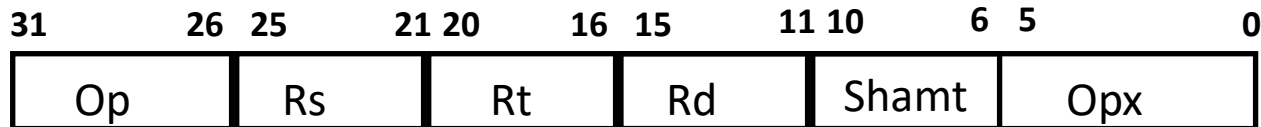
Pipelining: Basic and Intermediate Concepts



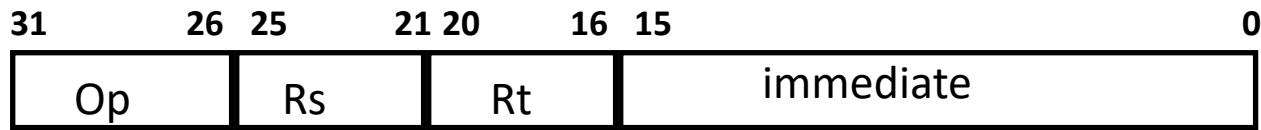


Example: MIPS (MIPS)

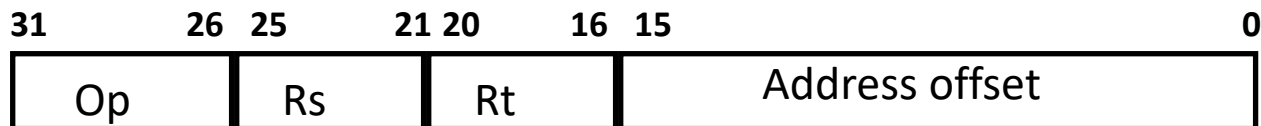
Register-Register (ADD, AND..)



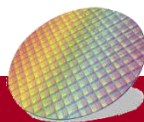
Register-Immediate (LD, SD)



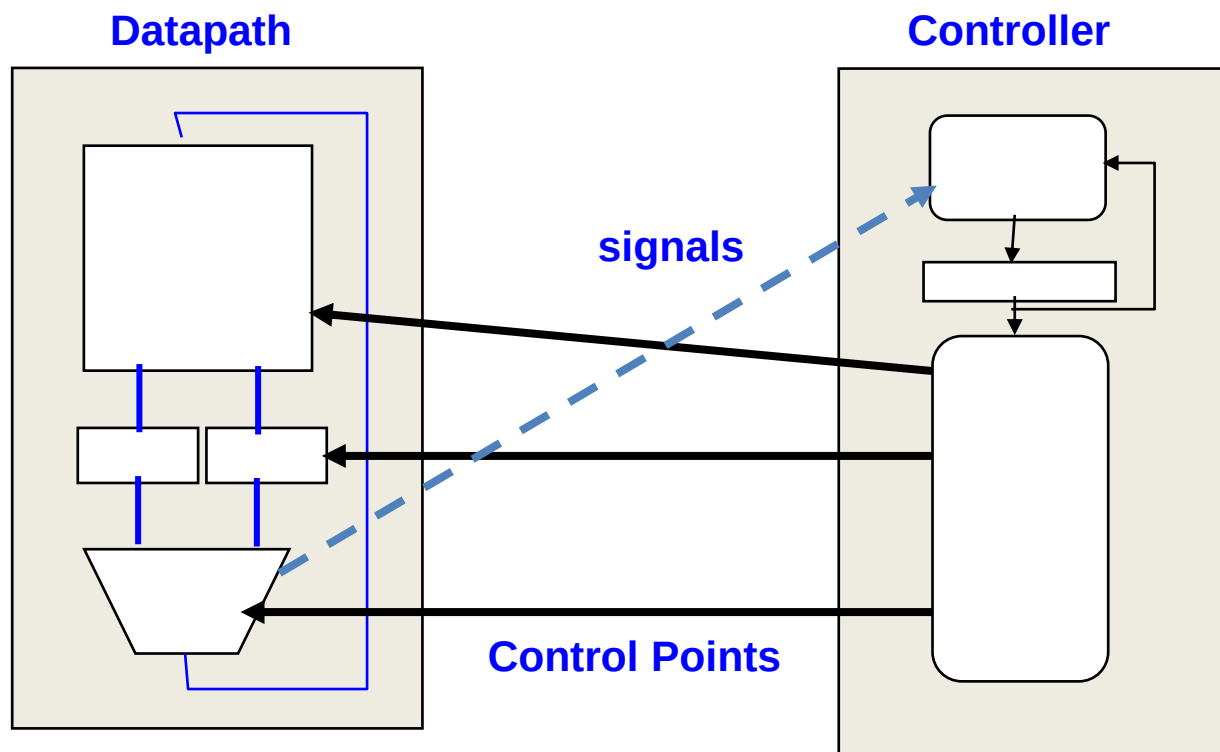
Branch (BEQ)



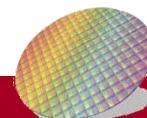
Jump / Call (J)



Datapath vs Control



- **Datapath**: Storage, FU, interconnect sufficient to perform the desired functions
 - Inputs are Control Points
 - Outputs are signals
- **Controller**: State machine to orchestrate operation on the data path
 - Based on desired function and signals



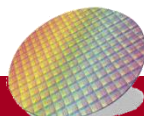
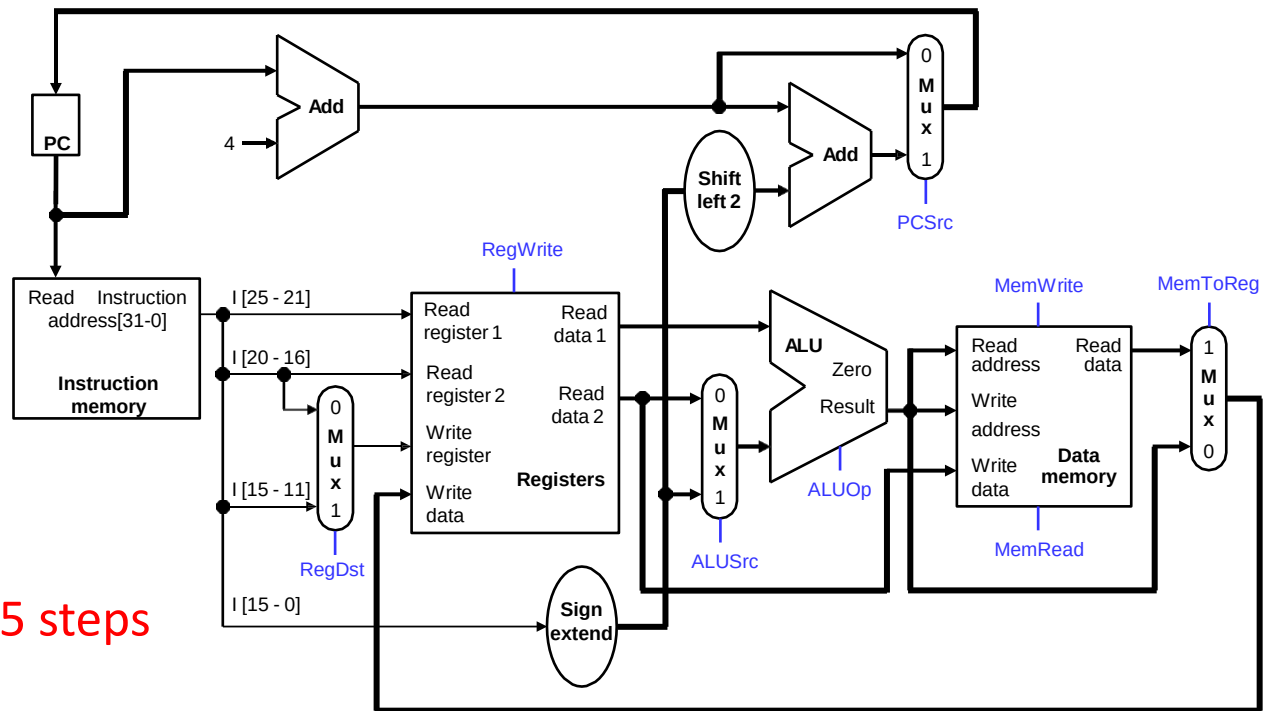


MIPS Datapath

- **Instructions**: subset of MIPS

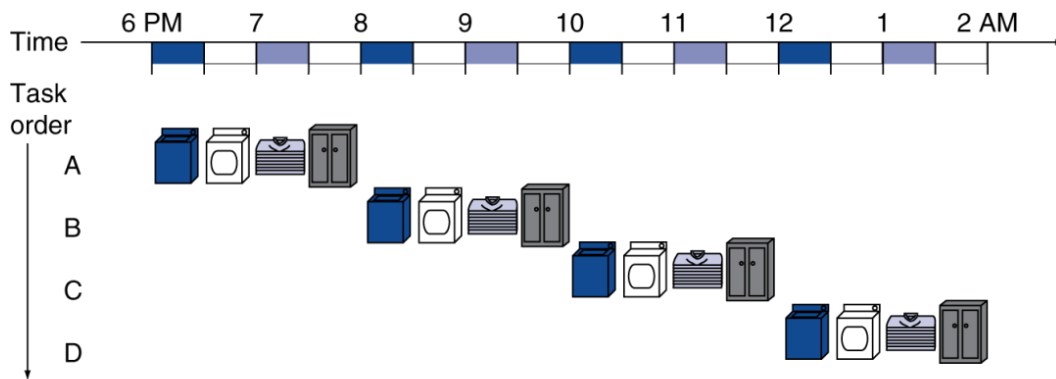
- Consists **load-store** word, **branch** and **integer** ALU operations

- Every instruction can be implemented in **at most 5 steps**
 - Instruction fetch (IF)
 - Instruction decode/register fetch (ID)
 - Execution/effective address cycle (EX)
 - Memory Access (MEM)
 - Write-back cycle (WB)



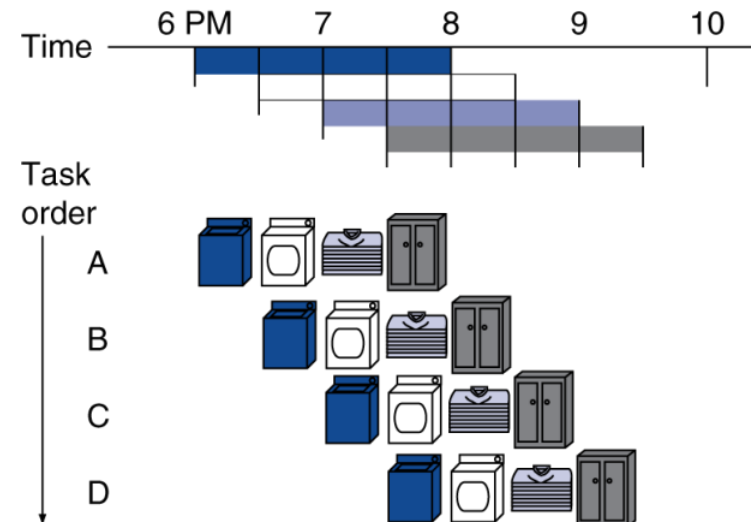
Introduction to pipelining

- **Pipelining**: Pipelining is an implementation technique whereby multiple instructions are overlapped in execution
- Like **assembly line**

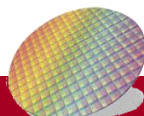


Assume four steps to do laundry, each one take 30min

- Time for four load = $2 \text{ hr} * 4 = 8 \text{ hr}$

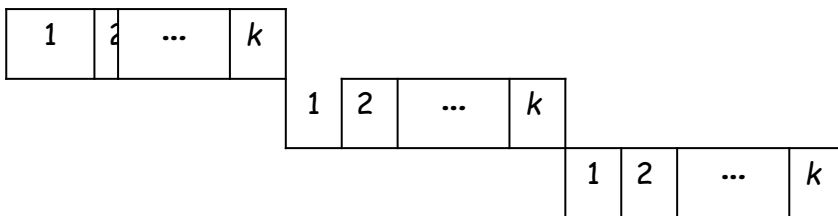


- Time for four loads
 - 3.5 hr
- Speedup = $8 / 3.5 = 2.3$



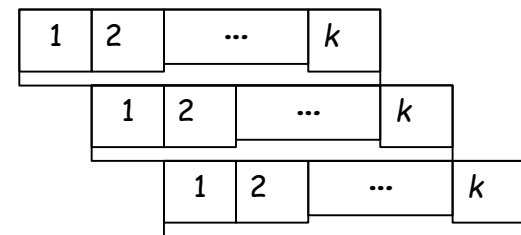
What is Pipelining?

- ❖ Consider a task that can be divided into k subtasks
 - ✧ The k subtasks are executed on k different stages
 - ✧ Each subtask requires one time unit
 - ✧ The total execution time of the task is k time units
- ❖ Pipelining is to **overlap** the execution
 - ✧ The k stages work in parallel on k different tasks
 - ✧ Tasks **enter/leave** pipeline at the rate of **one task per time unit**



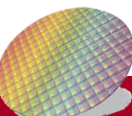
Serial Execution

One completion every k time units



Pipelined Execution

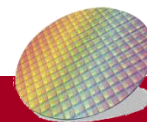
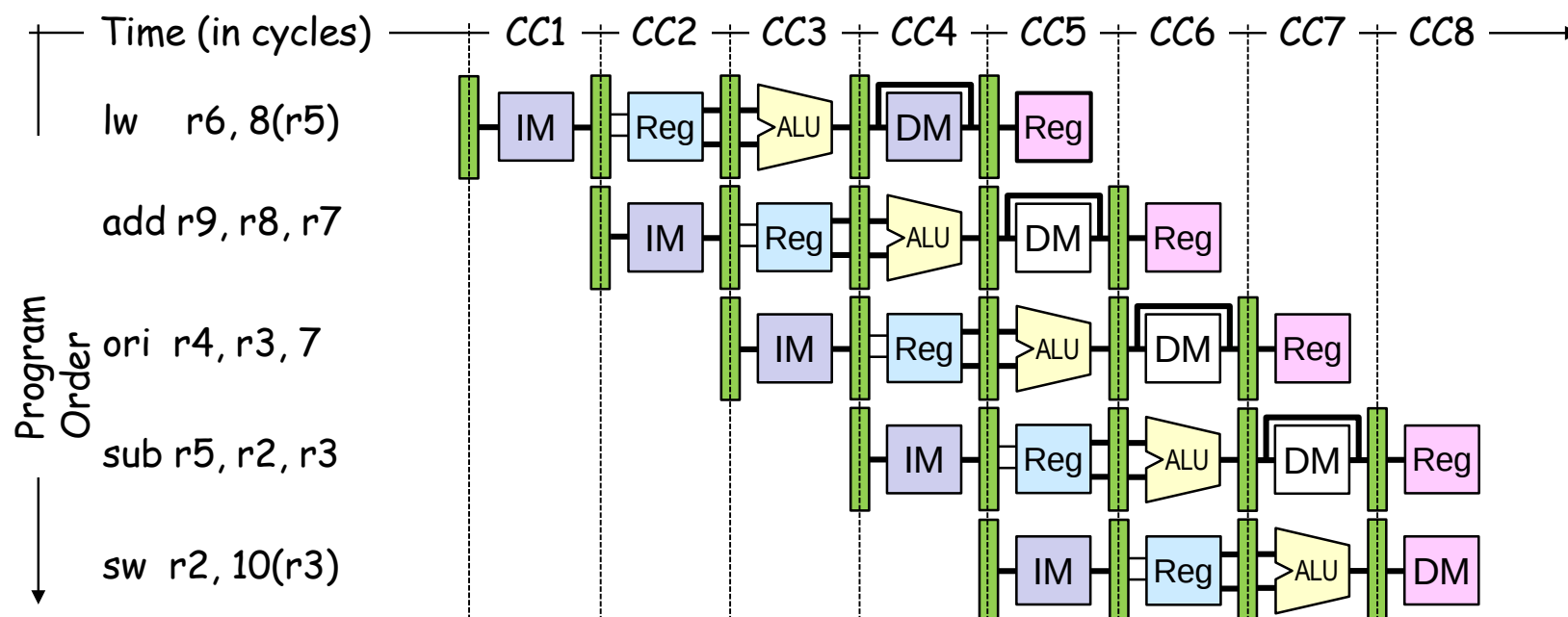
One completion every 1 time unit



Visualizing the Pipeline

❖ Multiple instruction execution over multiple clock cycles

- ❖ Instructions are listed in program order from top to bottom
- ❖ Figure shows the **use of resources** at each stage and each cycle
- ❖ **No interference** between different instructions in adjacent stages

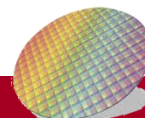
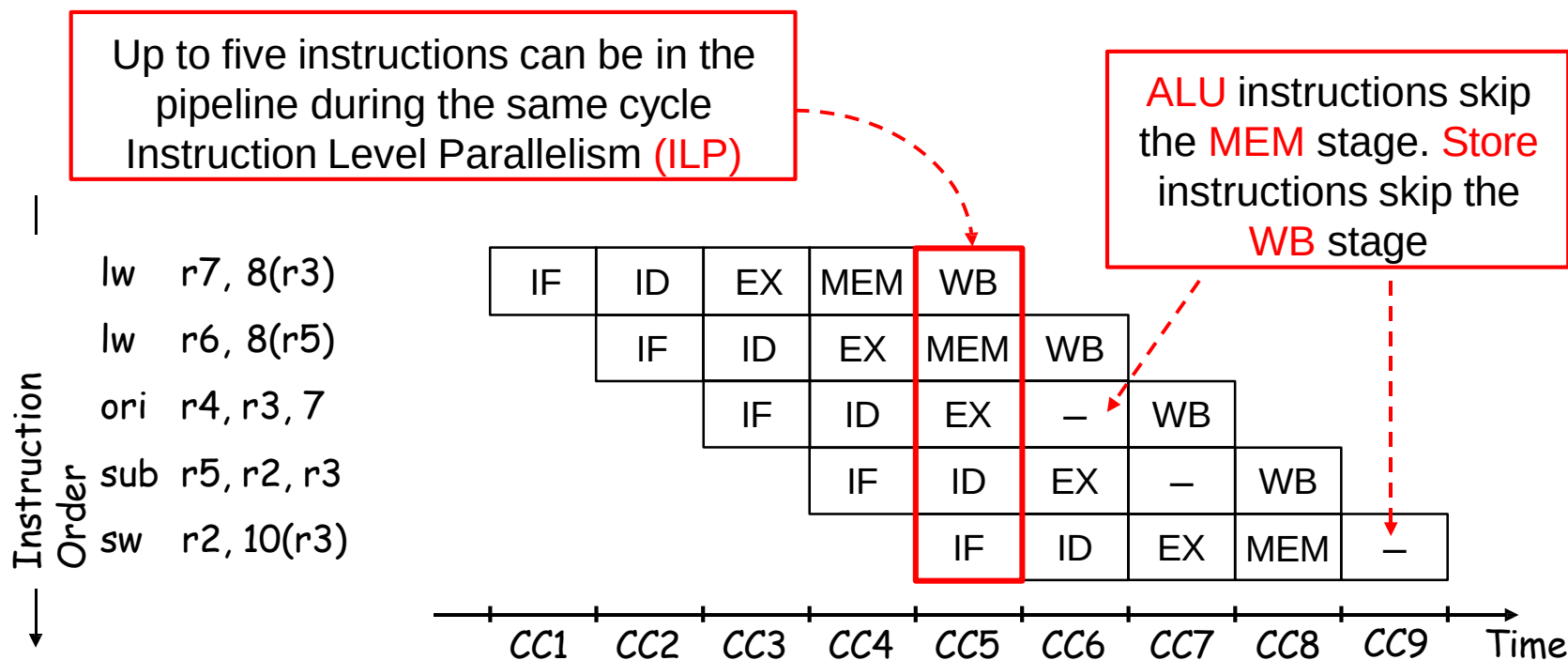


Visualizing the Pipeline (2)

❖ Time Diagram shows:

✧ Which instruction occupying what stage at each clock cycle

❖ Instruction flow is pipelined over the 5 stages



Example of Pipeline Performance

❖ Consider a 5-stage instruction execution pipeline ...

✧ Instruction fetch = ALU = Data memory access = 350 ps

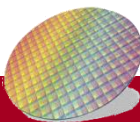
✧ Register read = Register write = 250 ps

❖ Compare single-cycle, multi-cycle, versus pipelined

✧ Assume: 20% load, 10% store, 40% ALU, and 30% branch

❖ Solution:

Instruction	Fetch	Reg Read	ALU	Memory	Reg Wr	Time
Load	350 ps	250 ps	350 ps	350 ps	250 ps	1550 ps
Store	350 ps	250 ps	350 ps	350 ps		1300 ps
ALU	350 ps	250 ps	350 ps		250 ps	1200 ps
Branch	350 ps	250 ps	350 ps			950 ps



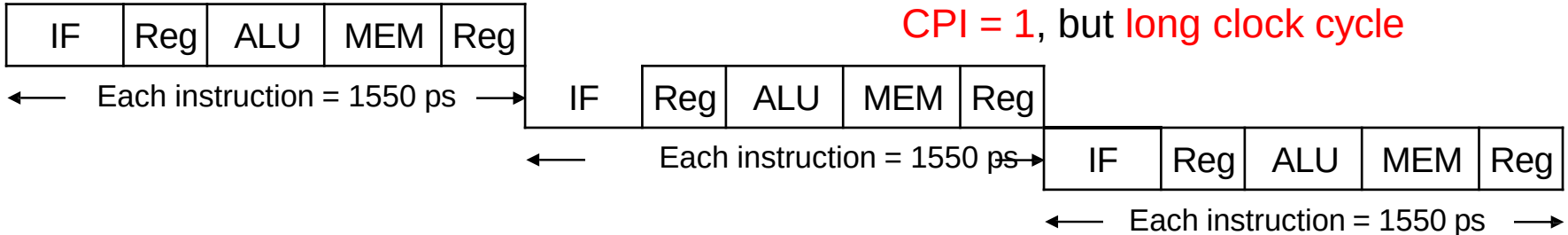


Single-Cycle, Multi-Cycle, Pipelined

Single-Cycle Execution:

$$T_{\text{clock}} = 350 + 250 + 350 + 350 + 250 = 1550 \text{ ps}$$

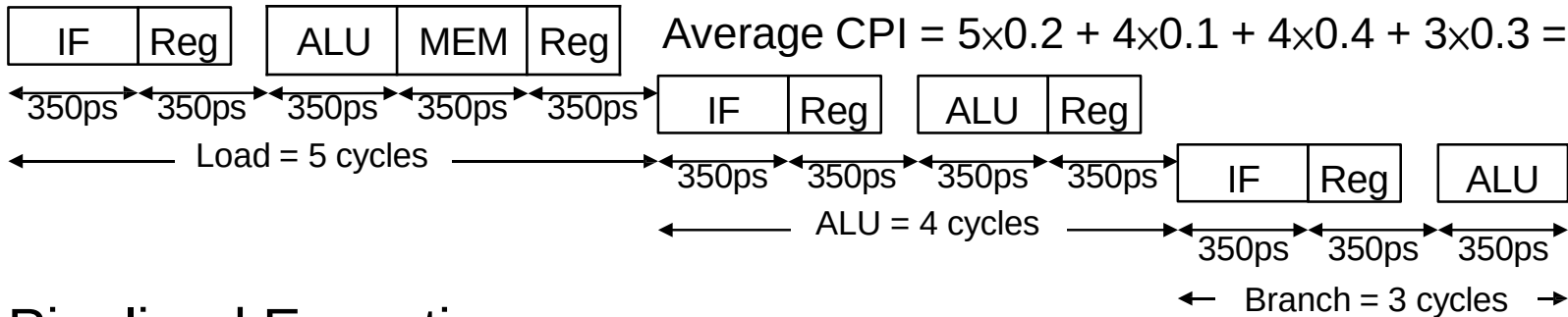
CPI = 1, but long clock cycle



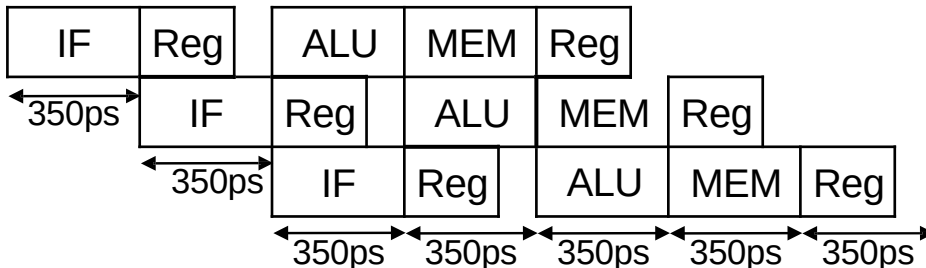
Multi-Cycle Execution:

$$T_{\text{clock}} = 350 \text{ ps}$$

$$\text{Average CPI} = 5 \times 0.2 + 4 \times 0.1 + 4 \times 0.4 + 3 \times 0.3 = 3.9$$



Pipelined Execution:

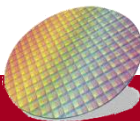


$$T_{\text{clock}} = 350 \text{ ps} = \max(350, 250)$$

One instruction completes each cycle

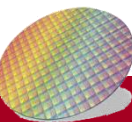
Average CPI = 1

Ignore time to fill pipeline



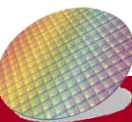
Single-Cycle, Multi-Cycle, Pipelined

- ❖ Single-Cycle CPI = 1, but long clock cycle = 1550 ps
 - ✧ Time of each instruction = 1550 ps
- ❖ Multi-Cycle Clock = 350 ps (faster clock than single-cycle)
 - ✧ But average CPI = 3.9 (worse than single-cycle)
 - ✧ Average time per instruction = $350 \text{ ps} \times 3.9 = 1365 \text{ ps}$
 - ✧ Multi-cycle is faster than single-cycle by: $1550/1365 = 1.14x$
- ❖ Pipeline Clock = 350 ps (same as multi-cycle)
 - ✧ But average CPI = 1 (one instruction completes per cycle)
 - ✧ Average time per instruction = $350 \text{ ps} \times 1 = 350 \text{ ps}$
 - ✧ Pipeline is faster than single-cycle by: $1550/350 = 4.43x$
 - ✧ Pipeline is also faster than multi-cycle by: $1365/350 = 3.9x$



Pipeline Performance Summary

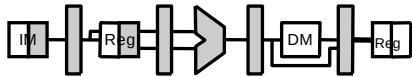
- ❖ Pipelining doesn't improve **latency** of a single instruction
- ❖ However, it improves **throughput** of entire workload
 - ✧ Instructions are initiated and completed at a higher rate
- ❖ In a **k -stage** pipeline, **k** instructions operate **in parallel**
 - ✧ Overlapped execution using multiple hardware resources
 - ✧ Potential speedup = **number of pipeline stages k**
 - ✧ Unbalanced lengths of pipeline stages reduces speedup
- ❖ Pipeline rate is limited by **slowest** pipeline stage
- ❖ Unbalanced lengths of pipeline stages reduces speedup
- ❖ Also, time to **fill** and **drain** pipeline reduces speedup



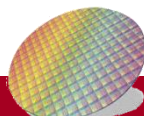
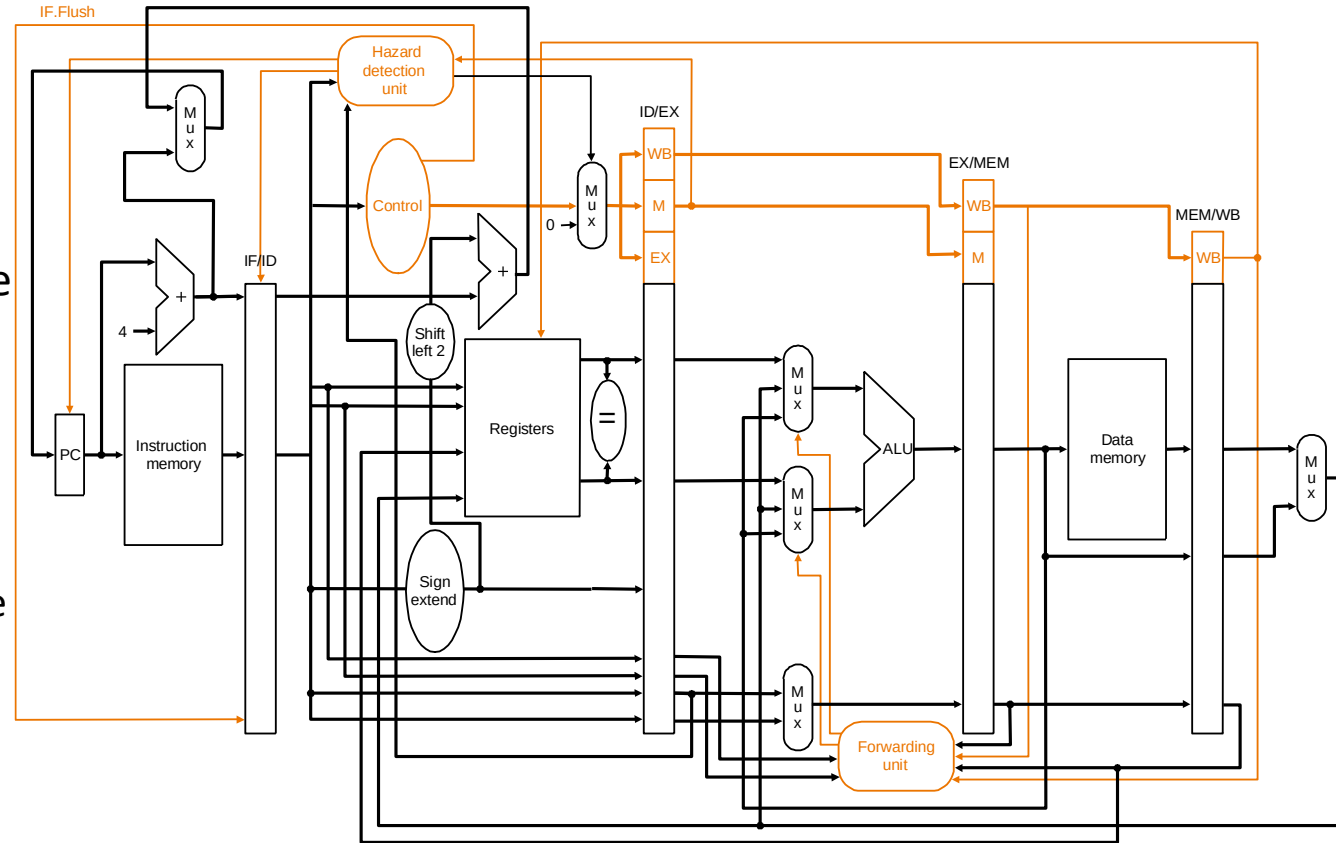


The Class Five-Stage Pipeline for a RISC processor

- Simplified version



- Each instruction takes at most 5 cycles to complete
- Separated **IM** & **DM**
- Register is used in two stages
 - **ID** & **WB**
 - Write in the first half cycle
 - Read in the second half cycle
- Branch is determined in **ID** stage
- Pipeline Register
 - To ensure instruction in different staged do not interfere with others
 - Called IF/ID, ID/EX, EX/MEM, MEM/WB



Pipeline Hazards

❖ **Hazard:** Situation that would cause incorrect execution

✧ If next instruction were launched during its designated clock cycle

1. Structural hazard

✧ Caused by hardware resource contention

✧ Using same resource by two instructions during same clock cycle

2. Data hazard

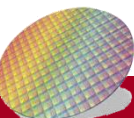
✧ An instruction may depend on the result of a prior instruction still in the pipeline, that did not write back its result into the register file

3. Control hazards

✧ Caused by instructions that change control flow (branches/jumps)

✧ Delays in changing the flow of control

❖ Hazards complicate pipeline control and limit performance



Structural Hazard

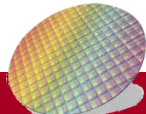
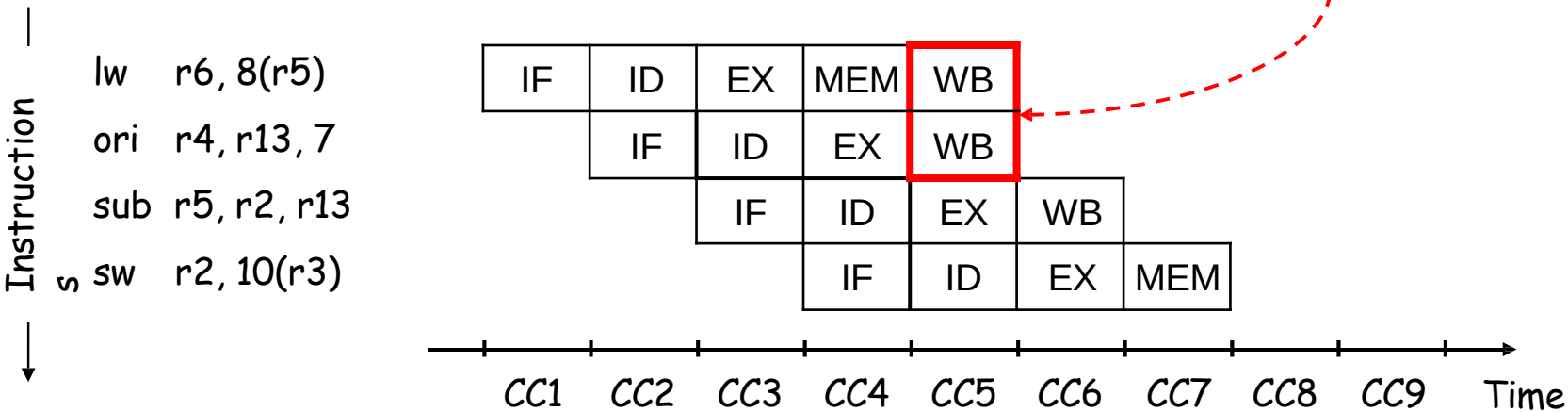
❖ Definition

- ✧ Attempt to use the same hardware resource by two different instructions during the same clock cycle

❖ Example

- ✧ Writing back ALU result in stage 4
- ✧ Conflict with writing load data in stage 5

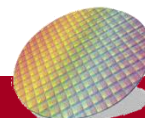
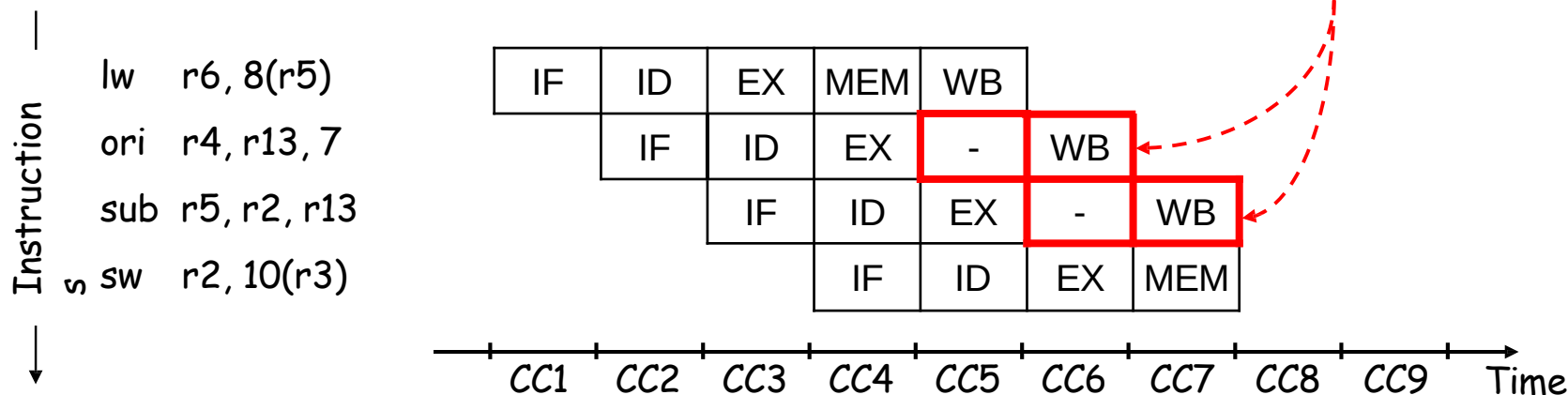
Structural Hazard
Two instructions are attempting to write the register file during same cycle



Resolving Structural Hazard

- ❖ Is it serious? Yes! cannot be ignored
- ❖ Solution 1: Delay access to resource
 - ✧ Delay Write Back to Stage 5
- ❖ Solution 2: Add more hardware
 - ✧ Two write ports for register file (costly)
 - Costly
 - Does not improve performance
 - One fetch → one completion per cycle

Resolving
Structural Hazard
Delay access
to register file
Write Back occurs
only in Stage 5



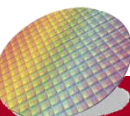
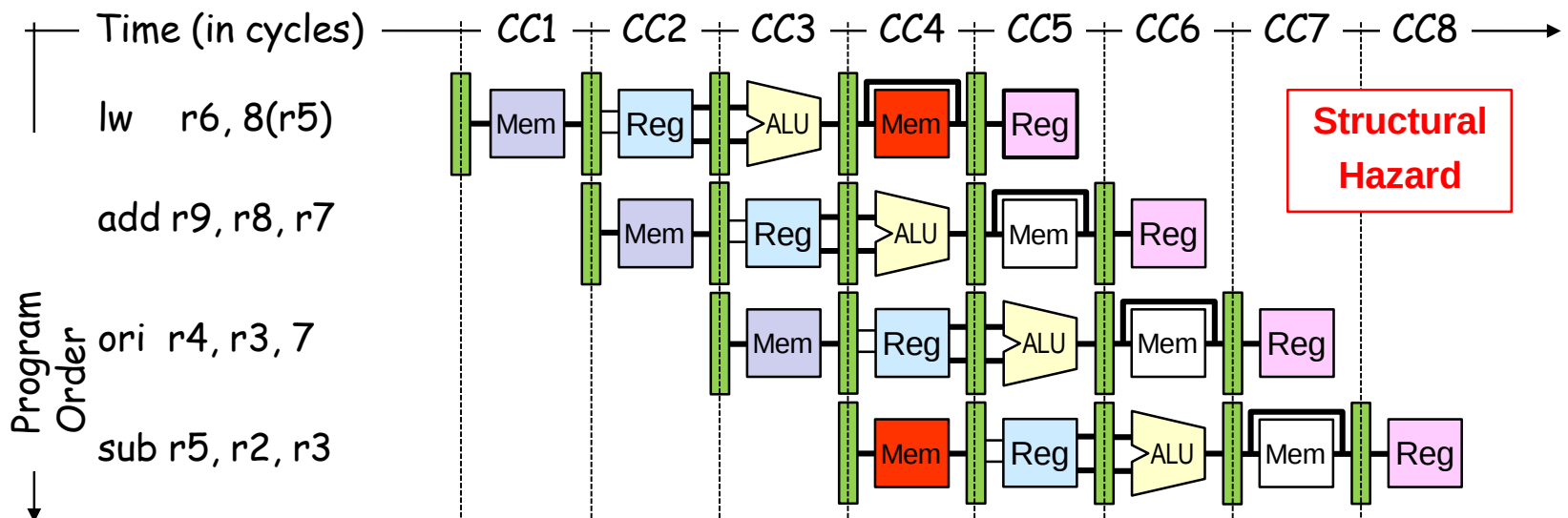
Example 2 of Structural Hazard

❖ One Cache Memory for both Instructions & Data

✧ **Instruction fetch** requires cache access each clock cycle

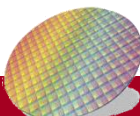
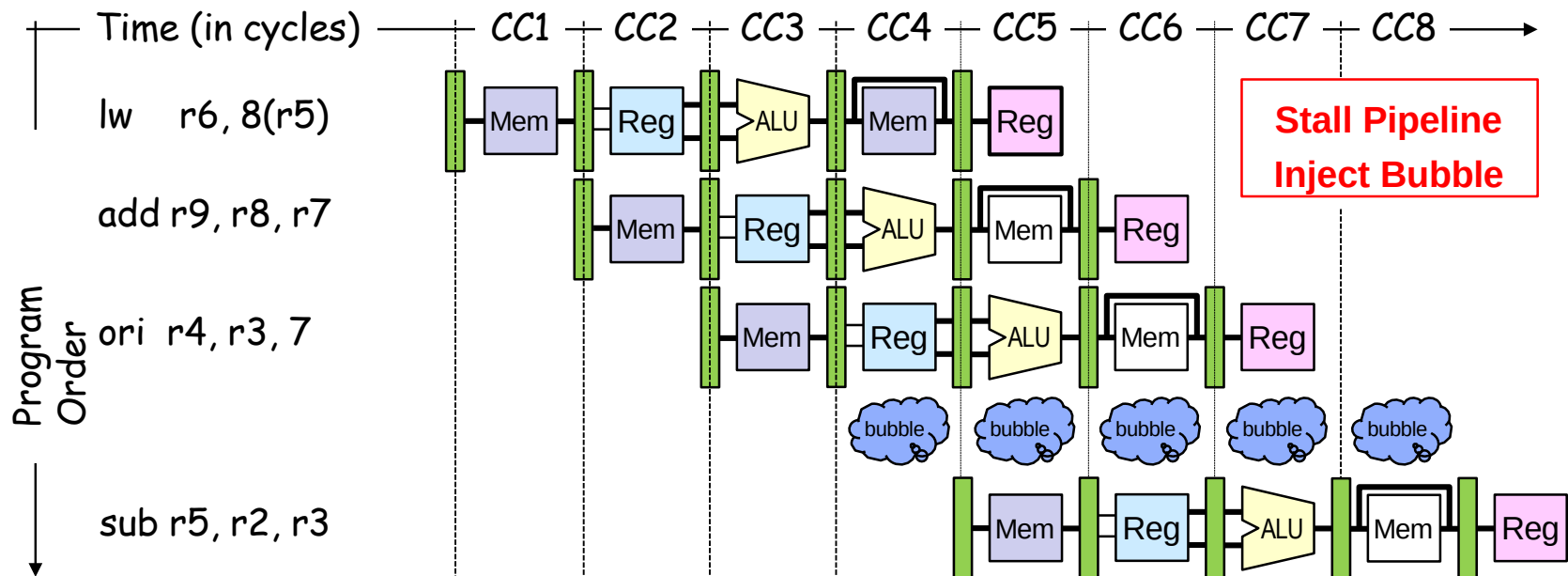
✧ **Load/store** also requires cache access to read/write data

✧ Cannot fetch instruction and load data if one address port



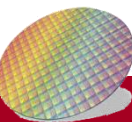
Stalling the Pipeline

- ❖ Delay Instruction Fetch → **Stall** pipeline (inject **bubble**)
 - ✧ Reduces performance: Stall pipeline for each load and store!
- ❖ Better Solution: Use **Separate** Instruction & Data Caches
 - ✧ Both caches are addressed **independently**: No structural hazard and No stalls



Resolving Structural Hazards

- ❖ **Serious Hazard:** structural hazard cannot be ignored
 - ✧ Can be eliminated with careful design of the pipeline
- ❖ **Solution 1: Delay Access to Hardware Resource**
 - ✧ Such as having all write backs to register file in the last stage, or
 - ✧ Stall the pipeline until resource is available
- ❖ **Solution 2: Add more Hardware Resources**
 - ✧ Add more hardware to eliminate the structural hazard
 - ✧ Such as having two **cache** memories for **instructions** & **data**
 - ✧ I-Cache and D-Cache can be addressed in parallel (same cycle)
 - Known as Harvard Architecture
 - ✧ Better than having **two address** ports for same cache memory



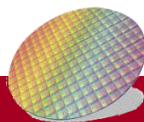
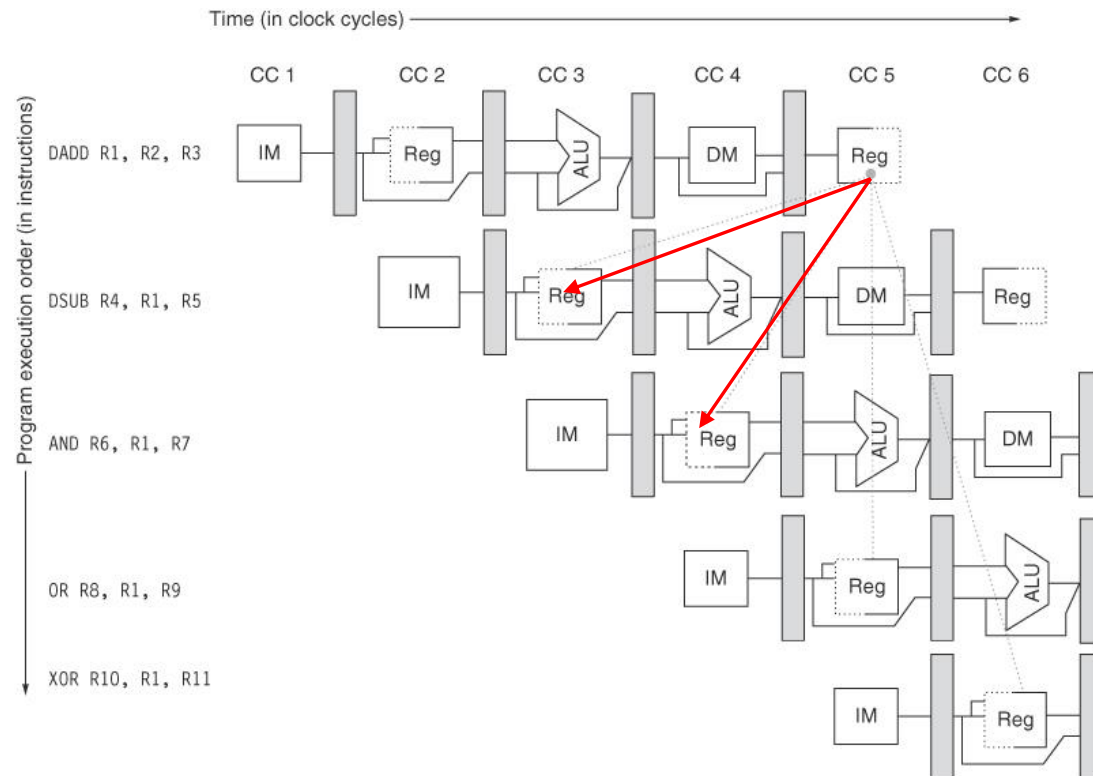


Data Hazard

- **Data hazard** occurs when an instruction depends on completion of data access by a previous instruction

```
DADD R1, R2, R3
DSUB R4, R1, R5
AND R6, R1, R7
OR R8, R1, R9
XOR R10, R1, R11
```

The result of DADD is written to Reg at CC5, but DSUB and AND need to read the data at CC3 and CC4



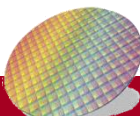
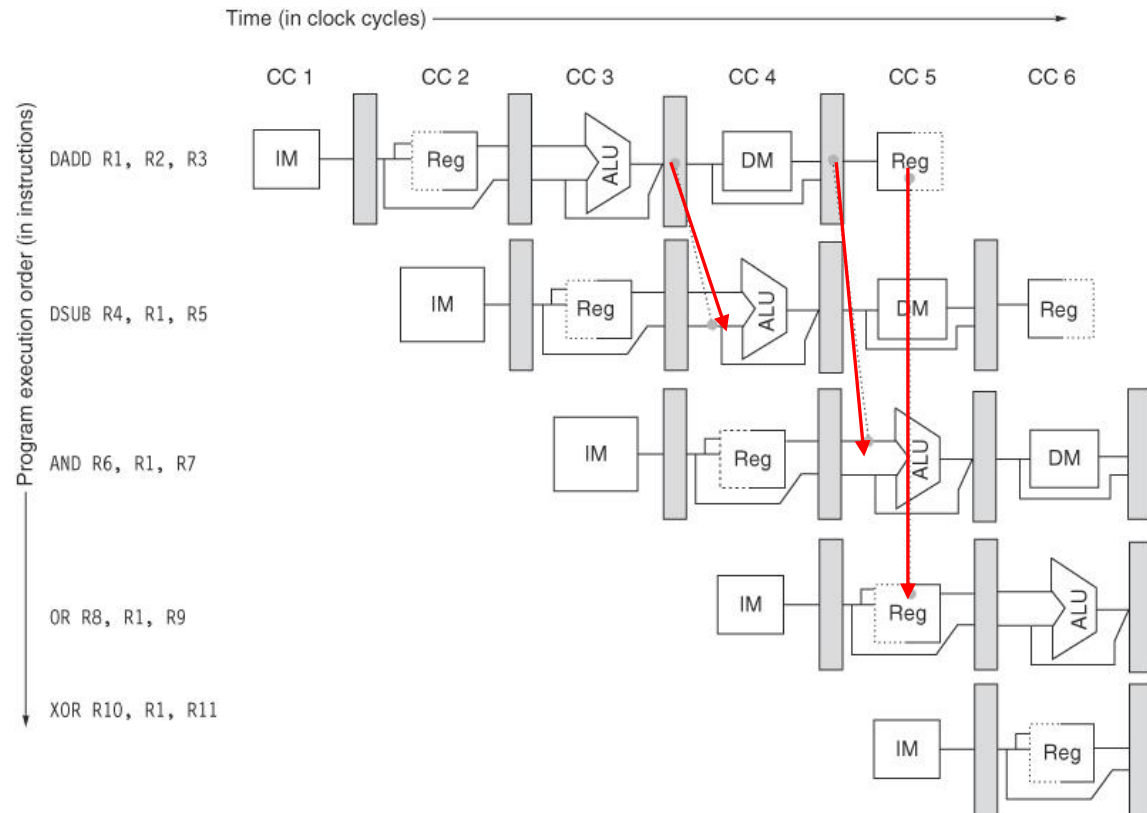


Minimizing Data Hazard Stalls by Forwarding

- **Forwarding**: forward data from pipeline register to reduce stalls

```
DADD R1, R2, R3
DSUB R4, R1, R5
AND R6, R1, R7
OR R8, R1, R9
XOR R10, R1, R11
```

- Data are forwarded from EX/MEM to ALU input
- Data are forwarded from MEM/WB to ALU input

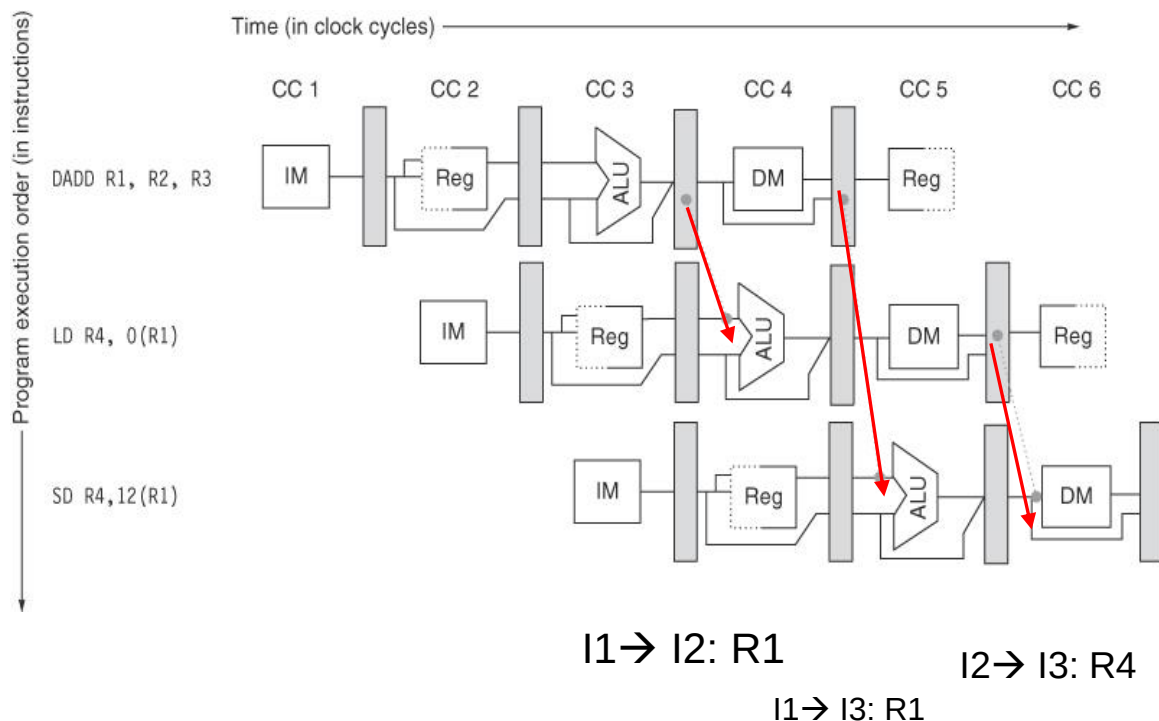




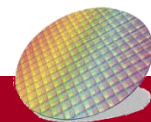
Forward Operands required by **stores** during MEM

- Data can be forwarded from one **output** of one unit to the **input** of another (no need to be the same unit)

```
DADD R1, R2, R3
LD   R4, 0(R1)
SD   R4, 12(R1)
```



- The result of the load is forwarded from the memory **output** to the memory **input** to be stored.
- ALU output is forwarded to the ALU input for the address calculation of both the load and the store.



Data hazards Requiring Stalls

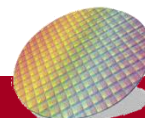
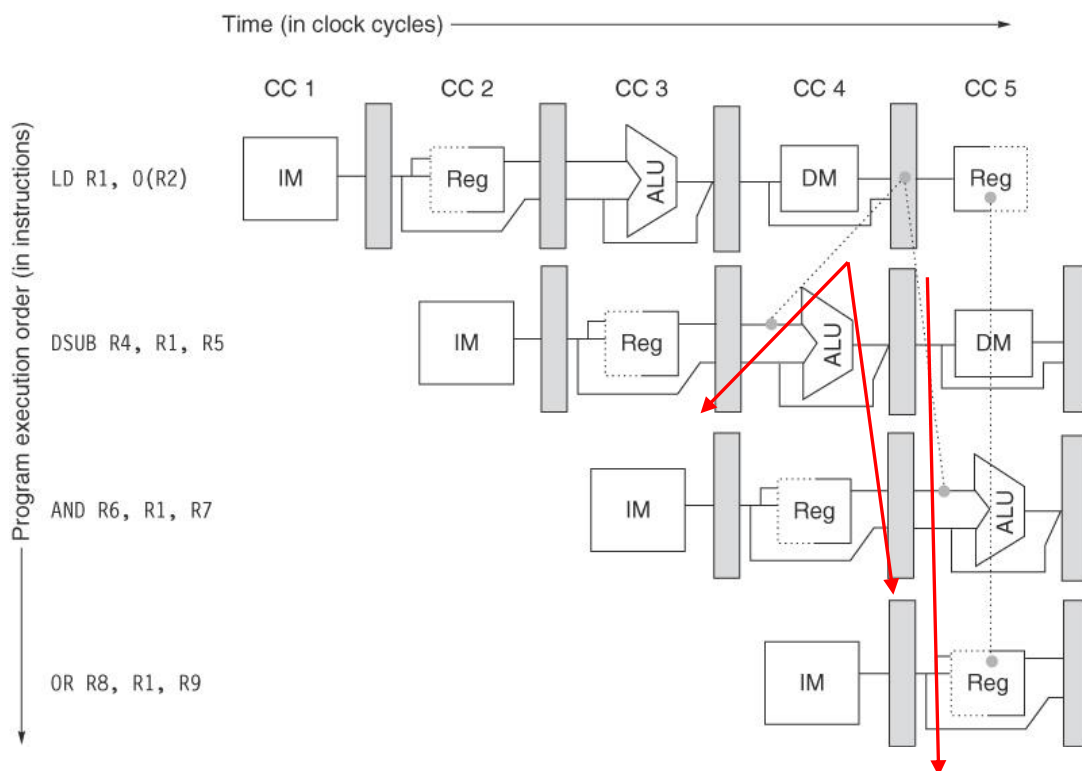
- Not all stalls can be avoided by forwarding

```
LD      R1, 0(R2)
DSUB    R4, R1, R5
AND     R6, R1, R7
OR      R8, R1, R9
```

- Result of LD is available until the end of CC4, but the data is needed by DSUB at the begin of CC4

=> Forwarding cannot eliminate the hazard

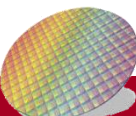
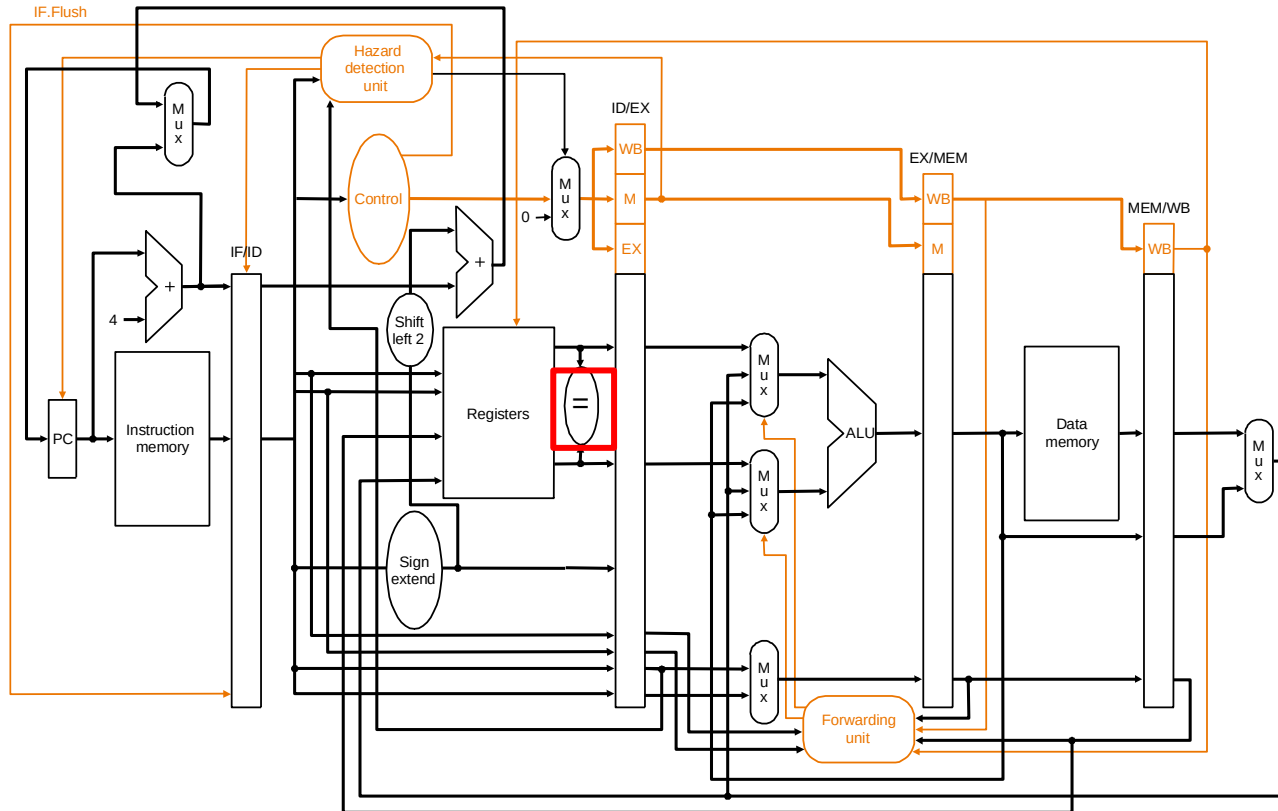
=> **Stall** is needed





Recall: Branch is determined at the end of ID Stage

- Equality test and calculate the target address are in ID stage



Control Hazard

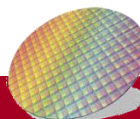
- A branch may change the PC to an address other than its current value+4
- If a branch changes the PC to its target address => branch is taken
- If it falls through (PC=PC+4) => not taken
- A branch causes a one-cycle stall in the five-stage pipeline

Inst. is fetched again.

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

If the branch is NOT taken, this is equal to

Branch inst. i	IF	ID	EX	MEM	WB		
nop							
Instruction i+1			IF	ID	EX	MEM	WB
Instruction i+2				IF	ID	EX	MEM
Instruction i+3					IF	ID	EX





Technique to Reduce Pipeline Branch Penalties

- Branches can be predicted to be **NOT TAKEN**
 - Simply continue as if the branch were not executed
 - If prediction is incorrect, execute branch target
- If branch outcome is **NOT TAKEN** then
 - Inst i+1** instructions can be executed
 - Do not convert **Inst i+1** into bubbles
 - No wasted clock cycles

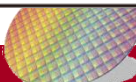
Branch inst. i
 Instruction i+1
 Instruction i+2
 Instruction i+3
 Instruction i+4

IF	ID	EX	MEM	WB				
	IF	ID	EX	MEM	WB			
		IF	ID	EX	MEM	WB		
			IF	ID	EX	MEM	WB	
				IF	ID	EX	MEM	WB

- If branch outcome is taken
 - Inst i+1 is converted to a **bubble**
 - Assume Inst i+2 is the correct inst.
 - 1 cycle penalty

Branch inst. i
 Instruction i+1
Branch Target
 Branch Target+1
 Branch Target+2

IF	ID	EX	MEM	WB				
	IF	nop	nop	nop	nop			
		IF	ID	EX	MEM	WB		
			IF	ID	EX	MEM	WB	
				IF	ID	EX	MEM	WB



Branch Hazard Alternatives

❖ Predict Branch Not Taken (previously discussed)

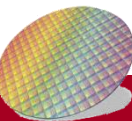
- ✧ Successor instruction is already fetched
- ✧ Do NOT kill instruction after branch if branch is NOT taken
- ✧ Kill only instructions appearing after Jump or taken branch (1-cycle penalty)

❖ Delayed Branch

- ✧ Define branch to take place **AFTER** the next instruction
- ✧ Compiler/assembler **fills the branch delay slot (only one slot)**

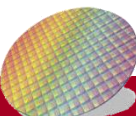
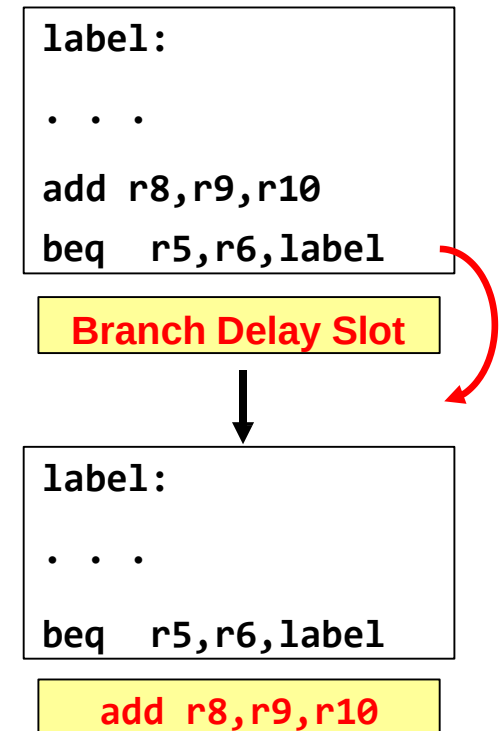
❖ Dynamic Branch Prediction

- ✧ Loop branches are taken most of time
- ✧ How to predict the branch behavior at runtime?
- ✧ Must reduce the branch delay to zero, but how?



Delayed Branch

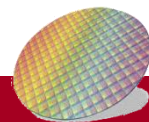
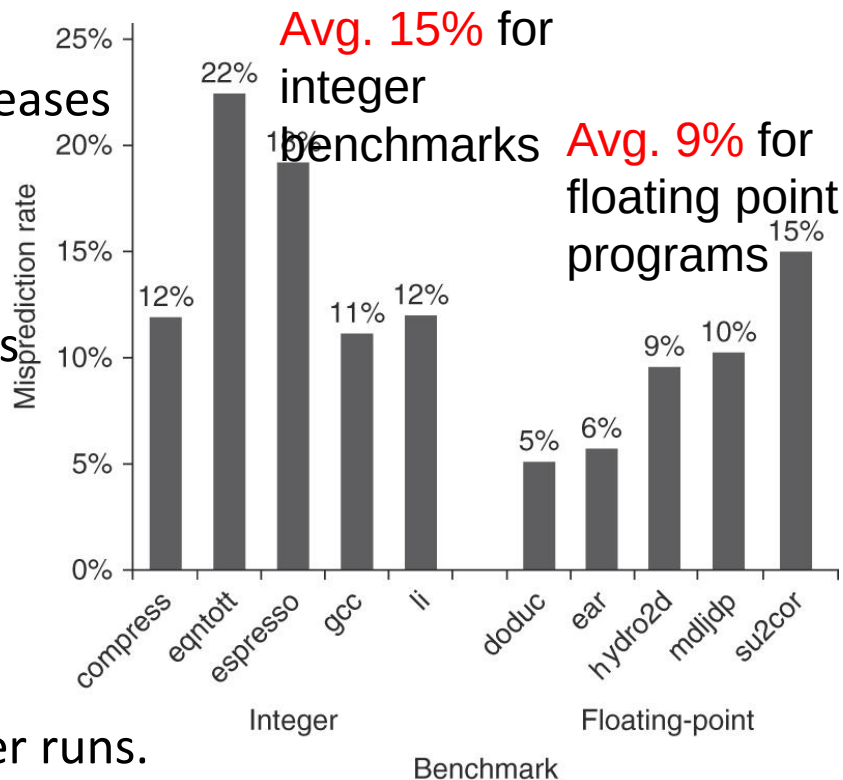
- ❖ Define branch to take place **after** the next instruction
- ❖ MIPS defines **one delay slot**
 - ✧ Reduces branch penalty
- ❖ Compiler **fills the branch delay slot**
 - ✧ By selecting an **independent instruction** from before the branch
 - ✧ Must be okay to execute instruction in the delay slot whether branch is taken or not
- ❖ If no instruction is found
 - ✧ Compiler fills delay slot with a NO-OP





Reducing Cost of Branches through Prediction

- More aggressive prediction methods are needed
 - Pipeline get deeper and branch penalty increases
- Two methods
 - **Static Branch Prediction (See below)**
 - **Dynamic branch prediction:** predict branches dynamically based on program behavior
- **Static Branch Prediction:**
 - Compile time technique
 - Use profile information collected from earlier runs.
 - Key observation: branch is often highly biased toward taken or untaken
 - E.g. predict not taken, delayed branch





Recall: Branch history table with 1-bit predictor

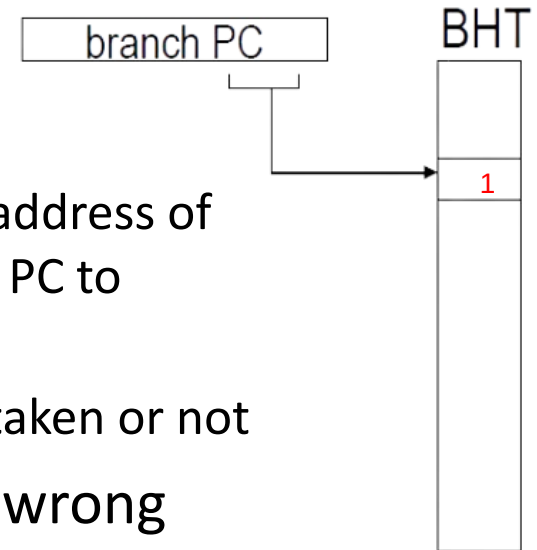
- Branch history table :

- Is a **table** of predictors
- a small memory indexed by the lower portion of the address of the branch instruction (Uses low-order bits of branch PC to choose entry)
- Contains **a bit** says whether the branch was recently taken or not

- No full address check => save HW, but may be wrong

- 1-bit predictor:

- When 0 => predict not taken,
 - When 1 => predict taken,
 - Change **states** when incorrectly predicted
- Has limited size
 - 1 bits by N (e.g. 4K entries)

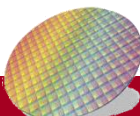


In **Fetch** phase of branch:

- Predictor from BHT used to make prediction

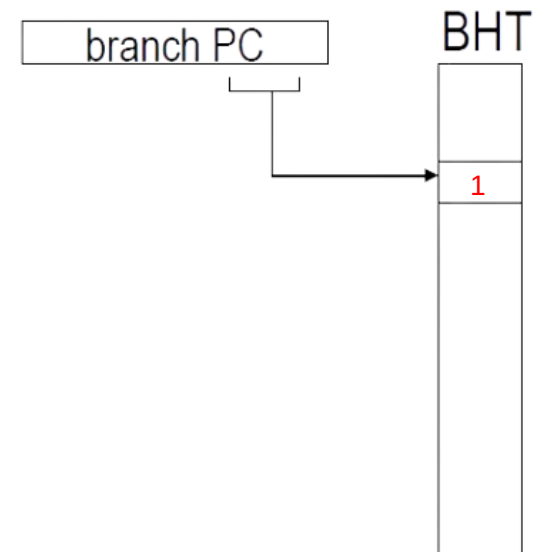
- When **branch completes**:

- Update corresponding Predictor



Issue of 1-Bit Predictor

- Two states: 0, 1,
 - 0=> predict **not taken**
 - 1=> prediction **taken**



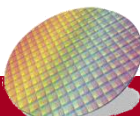
Execution Pattern

Predictor value at time of prediction

Predicted branch

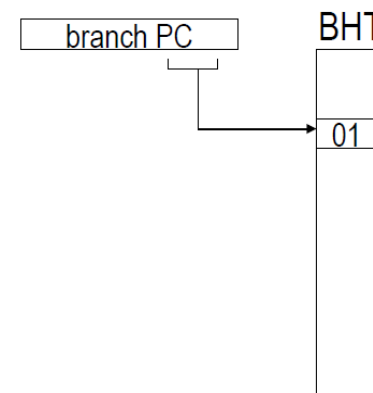
Prediction result in steady state (Correct, or incorrect)

T	T	N	T	T	T
0	1	1	0	1	1
N	T	T	N	T	T
I	C	I	I	C	C



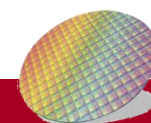
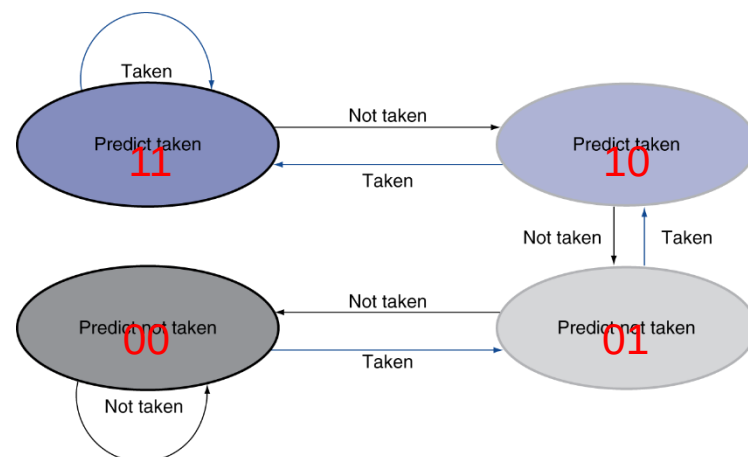
Recall: BHT with 2-Bit Predictor

- BHT in 2-bit predictor is a table of “Predictors”
 - 2-bit saturating counters indexed by PC address of Branch
- Four states: 00, 01, 10, 11
 - 00,01=> predict **not taken**, 10, 11=> prediction **taken**
- Only change prediction on **two** successive mispredictions



Execution Pattern	T	T	N	T	T	T	N	T
Predictor value at time of prediction	0	1	2	1	2	3	3	2
Predicted branch	N	N	T	N	T	T	T	T
Prediction result in steady state (Correct, or incorrect)	I	I	I	I	C	C	I	C

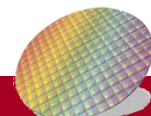
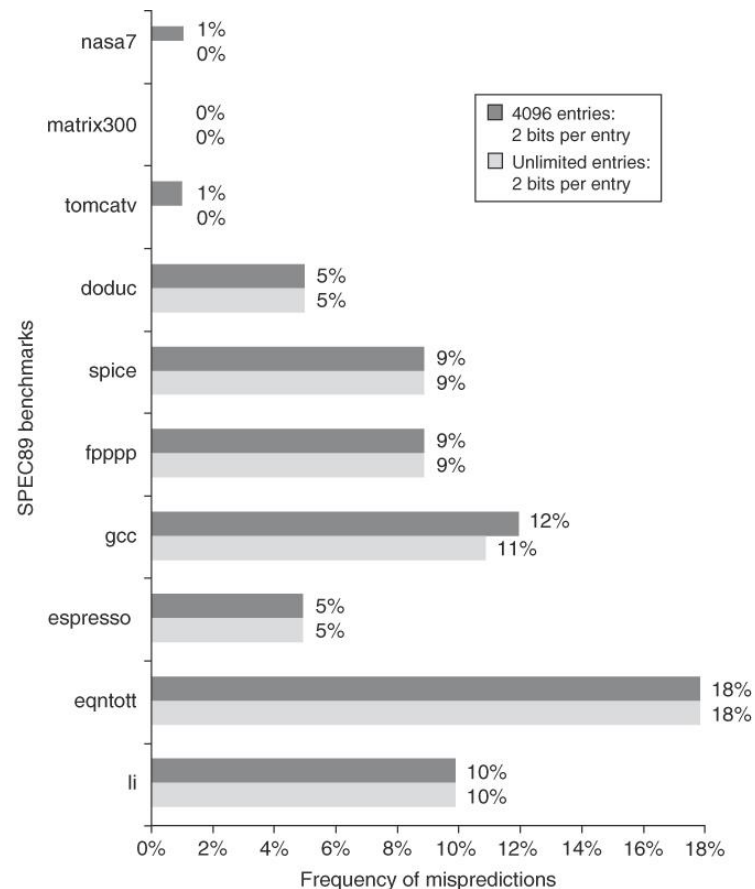
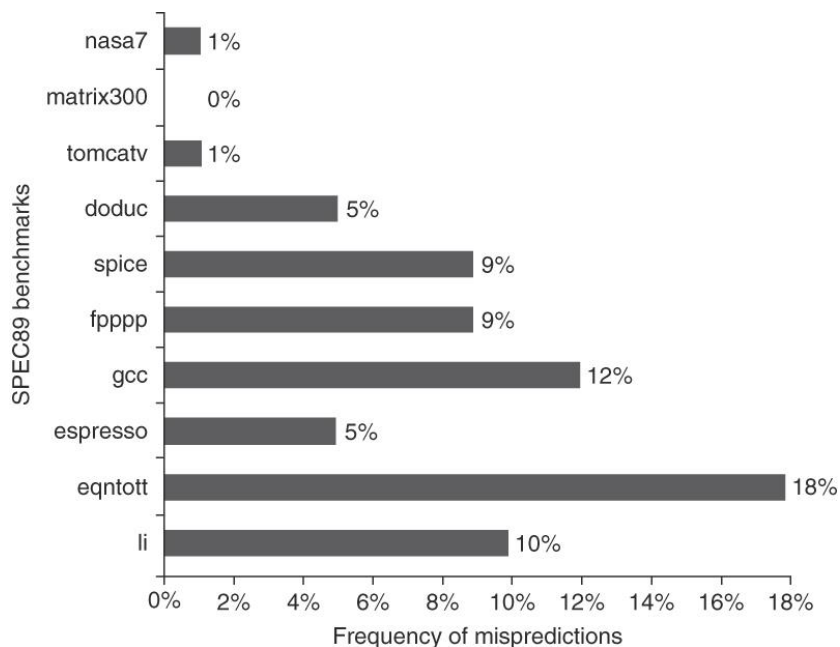
T	T	N	T	T	T	N	T
0	1	2	1	2	3	3	2
N	N	T	N	T	T	T	T
I	I	I	I	C	C	I	C





Prediction accuracy of a 4096-entry 2-bit prediction buffer for the SPEC 89 benchmark

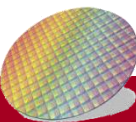
- Integer benchmark (gcc, espresso, eqntott, and li) has higher misprediction rate
- 4096-entry 2-bit prediction buffer has similar accuracy to infinite buffer
 - does not improve beyond 4K entries





Summary

- Read Appendix C1-C3
- Pipelining: Basic and Intermediate Concepts
 - 5 Stages: IF, ID, EX, MEM, WB
 - Pipeline register
- Three hazards
 - Structure, Data, Control
- Technique to reduce penalty of control hazard
- What Makes Pipelining Hard to implement (skipped)





成功大學

National Cheng Kung University

Backup Slides

